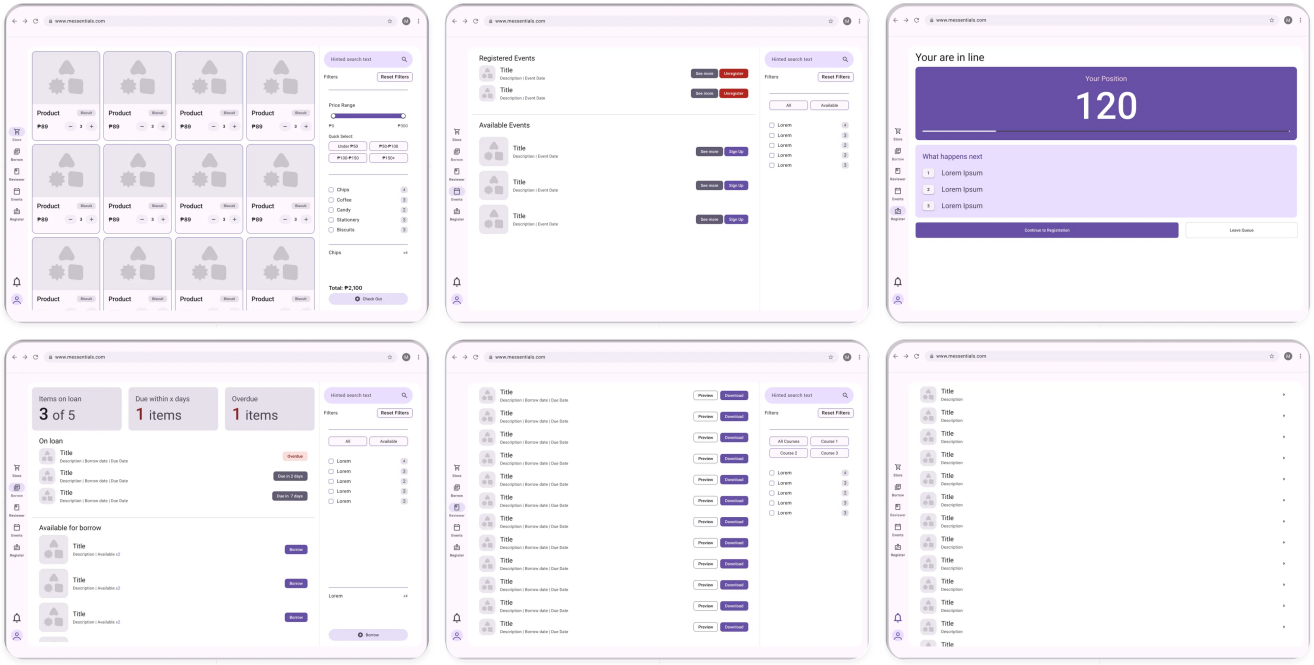


Our website uses most of its components and graphics from Material Design 3, Google's open-source design system for building usable products. This provides us a way to have a consistent set of interface components, colors, typography, shapes, and layout guidelines across our web applications.

Create a UI_DESIGN.md file that contains Theme token definitions (Tailwind classes, color palette), component structures (Sidebar, Header, Dashboard Grid, Modal), and direct links to Figma frames or embedded UI image paths. Assume this md file will be used to guide AI models in creating the website UI.

Created a file, shared a file, looked for files ›

I've written `UI_DESIGN.md`. It covers the color tokens (matched to your screenshots), a Tailwind config, the type scale, the five components you asked for, and a recipe for each of the six screens.

- **Placeholders to fill in:** I have no Figma links, so each frame URL is marked `TODO`. The image paths (`docs/ui/01-store.png` through `06-notifications.png`) are names I chose, and the screenshots aren't in the workspace yet. Save them under those names, or edit the paths

in the file.

- **Estimated colors:** the hex values are the Material Design 3 baseline purple scheme, matched by eye to your screenshots. Check them against your Figma styles.
- **Modal:** none of the screens show one. I specified it from the Material Design 3 dialog and listed where to use it (Leave Queue, Unregister, Check Out, Borrow, event "See more").
- **Typo in the design:** the Queue screen reads "Your are in line". The file says to use "You are in line" and flag it to your designer.
- **Header:** there is no in-app top bar in the screens. The file treats page headings as the header and tells models not to build the browser frame.

Next step: once you have the Figma frame URLs, replace the `TODO` entries in section 1.2.

assume KLP-1 is the updated version.

create a file `SPEC.md` (Functional & Technical Blueprint)

- Scope: Purpose, API routes, database schemas, file upload limits (e.g., 50MB PDF/Image max), and Redis BullMQ queue job handling.

ask questions first for clarification before putting it in the md if not explicitly stated in the files.

Some frameworks that we will be using: Svelte, tailwindcss

Weighing backend and database choices for the app ›

I've read both documents and treat KLP-1 as the current web version. Before I write `SPEC.md`, I

need four answers the files don't settle.

How should the backend be built, and which database should it use? The docs allow PostgreSQL or MySQL but do not pick one.

SvelteKit full-stack + PostgreSQL (Recommended)

How should login work? The docs say UP Mail through OAuth, but the ER diagram also lists username and password.

Google OAuth + email/password fallback

Where are uploaded files stored, and what are the size limits? Your note gave 50MB for PDF/Image.

Supabase for database (holding information, Access strictly via pooled connection strings), Cloudflare R2 for reviewer file storage, use native pdf preview

Which jobs should run through the Redis BullMQ queue? Select all that apply.

Email notifications, Borrow due-date reminders, Reviewer text extraction + indexing, Registration queue "you're next" alerts

Designing the reviewer indexing job and registration alerts ›

Thanks, that settles the stack. I'll write `SPEC.md` now and flag anything the files still leave open as an assumption.

Created a file, shared a file ›

I've written `SPEC.md` using your four answers and KLP-1 as the current version. It covers purpose and scope, API routes, database schema, uploads, and BullMQ jobs, plus roles, algorithms, and testing.

How your answers shaped it:

- **Backend and database:** SvelteKit full-stack with Postgres on Supabase. Everything uses pooled connection strings. The web app and worker use the transaction pooler (port 6543) with prepared statements off. Migrations use the session pooler.
- **Auth:** Google OAuth plus an email/password fallback.
- **Files:** Reviewer PDFs go to R2. Students preview them with the browser's built-in PDF viewer through a signed URL, with no PDF library. The file limit is 50 MB.
- **Queue:** All four jobs you picked are in: email notifications, borrow reminders, reviewer text indexing, and "you're next" queue alerts.

Section 14 lists 15 assumptions for you to confirm. These matter most:

1. **Station devices.** The files don't say how the store and borrowing stations log in. I treated them as normal browsers where an ID scan proves who is borrowing.
2. **Scanner type.** The files say ID scanner, RFID scanner, and QR in different places. I assumed a keyboard-style reader matched to a new `id_card_code` field, with QR used only to identify items.
3. **Image size limit.** I used 50 MB for images too, from your "PDF/Image" example. You may want a lower cap.
4. **Hosting, email provider, and Redis host.** None are named in the files, and the worker needs a long-running process.
5. **Loan rules.** Five active loans (from the "3 of 5" card), 7-day loans, and a 2-day reminder lead are my guesses.

I also added an hourly upload-cleanup job that you didn't select. Delete it if you don't want it.

Reply with answers to section 14 and I'll update the file.

1. Do we specifically need redis and bullmq or native svelte is fine for the job handling?
2. Can we switch instead to PostgreSQL + Svelte Polling for queue engine?
3. email notifications are hard to use, due to monetary restrictions are web push notifications viable aside from the notification tab on their account?

Weighing web push against in-app notifications for reminders ›

1. Do we need Redis and BullMQ?

No. SvelteKit has no job system of its own. It only handles requests and responses. You could run timers inside the SvelteKit server, but they are lost on every restart. They also run twice if you have two server instances, and they don't work on serverless hosting. What you need is a *durable* job store, and it doesn't have to be Redis. At your scale (one department) Postgres is enough, and

it removes one service to pay for and hand over.

2. Can we use PostgreSQL + polling instead?

Yes. Two parts of the spec are involved:

- **Registration queue:** it already lives in Postgres (`queue_entries`, with an atomic ticket counter), and the Register page already polls every 10 seconds. Redis was never part of it. No change is needed.
- **Background jobs:** I'd move these to a Postgres `jobs` table. A worker claims rows with `FOR UPDATE SKIP LOCKED`, and each row has `run_after`, `attempts`, and `last_error`. You can build this yourself (about 100 lines) or use the `pg-boss` library. Avoid `graphile-worker`, because it relies on `LISTEN/NOTIFY`, which your transaction pooler doesn't support. I'd check `pg-boss` against the pooler before committing to it.

Most of the jobs get simpler without Redis:

- **Borrow reminders:** use a scheduled sweep instead of delayed jobs. Add `reminded_due_soon_at` and `reminded_overdue_at` columns to `borrow_records`. Every hour, the sweep finds loans due soon or overdue that haven't been reminded yet. Nothing needs to be cancelled on return, and nothing is lost if the system restarts.
- **"You're next" alerts:** compute them inside the transaction when a manager advances the queue. No job is needed.
- **Reviewer text extraction:** this still needs a job, because a 50 MB PDF is too slow for a request.
- **Where the worker runs:** either a small always-on process, or Supabase `pg_cron` calling a protected SvelteKit endpoint every minute. Serverless time limits could cut off long PDF extractions, so the always-on process is safer.

3. Are web push notifications viable?

Yes. They cost nothing per message (the browser makers run the delivery). You need a service worker, a VAPID key pair, the `web-push` library, and a `push_subscriptions` table with one row per device. SvelteKit supports service workers directly.

Limits to plan for:

- Students must click "Allow". Anyone who declines gets only the in-app tab.
- iPhones only support push when the site is added to the Home Screen as an installed web

app (iOS 16.4 and later).
- Delivery isn't guaranteed. Expired subscriptions come back as 404 or 410 errors, and the server should delete them.
- Keep the in-app notification list as the source of truth, with push as a nudge on top.

One side effect: the email/password fallback uses email for verification and password reset. Without email, that fallback needs another approach.

My recommendation is Postgres jobs, web push plus in-app notifications, and no Redis or email. I can update `SPEC.md` to match. Two choices first:

1. For the password fallback, should I drop it and use Google-only login, or keep it with admin-assisted password resets and no email verification?
2. Should I use `pg-boss` or a small custom jobs table?

1. keep the current Auth and email/username + password, suppose the students need to create an account first (no accounts available on the get go), how would you implement it?
2. custom jobs table
3. package manager is going to be pnpm
4. can you check if the relational schema normal form is at the best state for the implementation of the website? dont change anything for this yet, just verify

Assessing location and status fields for redundancy across tables ›

I haven't changed `SPEC.md` yet. Below are the sign-up design (1), the notes on your other answers (2 and 3), and the normal form check (4).

1. Account creation with no email

Without email, you can't prove a person owns an address. The design below handles that with a verification step based on Google sign-in or an in-person check.

First account. Nobody can approve the first admin, so seed it with a script: `pnpm seed:admin`. It reads an email and a temporary password from the terminal. After that, admins grant volunteer roles in the app.

Two ways to sign up

- **Google (UP Mail):** the account is created already verified, because Google proves the address.
- **Username + password:** the account is created as *unverified*.

Sign-up steps

1. The form asks for email (domain must be `up.edu.ph`), username, password, first name, last name, student number, and year standing.
2. Cloudflare Turnstile (free) blocks bots. This matters more because there is no email gate.
3. The server hashes the password with argon2id and creates the user as unverified.
4. The server creates 8 one-time **recovery codes** and shows them once. The student must save them.
5. The student is logged in with limited access.
6. Login takes "email or username" in one field.

Becoming verified

- Signing in with Google using the same email verifies the account. If a password account already exists for that email and is unverified, Google wins: link the account, then wipe the old password and sessions. This stops someone from squatting on another student's email.
- A volunteer checks the student's physical UP ID, clicks "Verify", and scans the card. That also fills `id_card_code`.

What unverified accounts can do (my proposal)

Feature	Unverified	Verified
Browse, store checkout, reviewers, notifications, messages	Yes	Yes
Borrow items	No	Yes (and ID scan)
Register for events	No	Yes
Join registration queue	No	Yes

Password reset without email

- **Recovery code:** the student enters their identifier, one code, and a new password. The code is marked used. All sessions are revoked.
- **No codes left:** a volunteer checks the ID in person and sets a temporary password. It is shown once, and `must_change_password` forces a change at the next login.

Schema and route changes this needs

- `users`: add `username citext unique`, `verified_at`, `verified_by`, `must_change_password`. Remove `email_verified_at`.
- New table `recovery_codes` (user_id, code_hash, used_at). Drop `email_tokens`.
- Routes: `POST /auth/register`, `POST /auth/login` (with `identifier`), `POST /auth/recovery/reset`, `POST /admin/users/:id/verify`, `POST /admin/users/:id/temporary-password`.

2 and 3. Custom jobs table and pnpm

Both are noted. pnpm was already the assumed package manager. I'll size the jobs table (`run_after`, `attempts`, `locked_at`, `last_error`, claimed with `FOR UPDATE SKIP LOCKED`) when I update the spec.

4. Normal form check (no changes made)

Result: the schema is in 3NF or BCNF for all entity tables, with a few deliberate exceptions listed below. It is good enough to build on.

- **2NF:** every composite-key table (`volunteer_roles`, `notification_settings`, `*_recipients`, `reviewer_bookmarks`, `reviewer_tag_map`) has columns that depend on the whole key.
- **4NF and 5NF:** the junction tables hold one relationship each, and there are no three-way relations, so nothing to split.
- **BCNF:** every determinant is a key in `queue_entries`, `borrow_records`, `event_registrations`, `uploads`, and `users`.

Findings, highest value first

#	Table.column	Issue	Verdict
1	`products.name_normalized`, `borrowables.name_normalized`	Derived from `name`, so `id → name → name_normalized` is a transitive dependency (breaks 3NF). If code forgets to update it, search breaks.	Fix. Use a generated column or an expression index on `lower(name)`.
2	`events.registrant_count`	Duplicates a count of `event_registrations`. It can drift.	You already lock the event row at registration, so `COUNT(*)` under that lock is safe and cheap here. Drop it unless it proves slow.
3	`event_registrations.answers` (jsonb)	A repeating group (breaks 1NF), and it has no link to `event_prompts`.	Normalize to `event_registration_answers(registration_id, prompt_id, answer)` if prompts can change after people register.
4	`reviewer_tags.course`	Mixes two ideas (tag and course). The text repeats, so renaming a course touches many rows.	Add a `courses` table with `reviewers.course_id` or a link table.
5	`category` text on `borrowables`, `events`	No normal form violation, but free text lets “Utilities” and “utilities” split the filter counts. The store already uses a lookup table.	Use lookup tables for consistency.
6	`registration_sessions.location` and `advisers.location`	The same fact can live in two places.	Keep one.
7	`borrowable_units.status = 'borrowed'`	Duplicates “an open loan exists”.	Keep it, on purpose. It is updated under a row lock, and the partial unique index guards it.

#	Table.column	Issue	Verdict
8	`purchases.total`	Derived from the line items.	Keep it. Purchases are never edited after insert.
9	`reviewers.authors text[]`, `borrowables.components jsonb`, `events.participation_constraints jsonb`	Not atomic (1NF).	Keep them if you only display them. Normalize if you need to filter by author or component.
10	`users.year_standing`	It goes stale each year.	Store an entry year instead, or update it each semester.
11	`notifications.email_status`, `notification_settings.email`	These become dead columns now that email is dropped.	Replace with a `push_subscriptions` table when I update the spec.

Smaller points: `purchase_items` has no unique key on `(purchase_id, product_id)`. `event_prompts` has no unique key on `(event_id, position)`. `audit_logs` is intentionally loose (polymorphic target, jsonb metadata).

Items 1, 2, 4, and 6 are cheap and worth doing now. The rest depend on how you will query the data.

Should I update `SPEC.md` with the new auth flow, Postgres jobs table, web push, and these schema fixes? If so, tell me which of the 11 findings to apply.

apply all changes, create a separate Changes.md file that highlights all revisions made from the original KLP{,1} files. Inside Changes.md make sure to be very thorough on original, the change, and reason for change, also include a fallback step if want to return to the original. Also take these new considerations into account